

Project Tricorder Mk I: A Microkernel Approach to Multimodal Edge-AI Systems

1. Architectural Philosophy: The Hybrid Microkernel Model

The Project Tricorder Mk I architecture adopts a **Hybrid Microkernel** paradigm, meticulously designed to maintain deterministic performance in high-stakes edge environments. In this model, the Android Kotlin wrapper functions as the privileged kernel space and Hardware Abstraction Layer (HAL). It manages low-level resource contention and provides a **System Call Interface (SCI)** via **TricorderBridge.kt**, allowing the platform-agnostic "User-Space" logic—the **ui-shell** and **tricorder-core**—to access native hardware services. To ensure system stability, the architecture adheres to the microkernel principle of minimality. Heavy, non-deterministic computations are offloaded to isolated daemons, but the system distinguishes between two tiers of execution:

1. **Kernel-Resident Inference:** High-efficiency models like **YAMNet** (Acoustics), **BirdNET** (Avian classification), and **iNaturalist** (Flora/Fauna) run natively within the Kotlin "Kernel" using TFLite. This minimizes the latency of frequent context switches between the native layer and user-space.
2. **User-Space Microservices:** Massive multimodal models—**Gemma**, **SmoIVLM**, **SD Turbo**, and **Kokoro**—are isolated as Termux-resident daemons reachable via loopback. This isolation protects the primary system from the significant memory overhead and potential crashes associated with large-scale GGUF and ONNX weights.

Core Kernel Services

Exposed via the SCI, these services are mapped to the **tricorder-core** "Brain":

- **Imaging:** Camera management for **Sensor** and **Sentry** modes.
- **Acoustics:** MIC access and raw PCM processing for **AudioSense.kt**.
- **Positioning:** GPS/GNSS clock interpolation and ground speed telemetry.
- **Environmentals:** Barometric pressure, ambient temperature, and humidity sensors.
- **Inertial Measurement Unit (IMU):** High-frequency gravity, accelerometer, and rotation vector data.

2. The System Spine: Pub/Sub IPC via eventBus.mjs

The internal communication of Project Tricorder is governed by **eventBus.mjs**, which serves as the central Inter-Process Communication (IPC) mechanism. This "System Spine" enforces a non-negotiable architectural rule: **UI, providers, and engines never touch each other directly**. Interaction occurs exclusively through asynchronous, non-blocking bus events. This decoupling is critical for managing the transition between the high-frequency native sensor callbacks—such as the **120ms IMU pushes** in **MainActivity.kt**—and the Javascript-based logic. By using a pub/sub pattern, the system maintains a responsive UI thread while heavy model responses are processed in the background.

Primary Event Types

- **SENSOR_DESCRIPTION** : Dispatched during **Scan Loops** to provide model-derived metadata of sensor input.
- **ALERT_RAISED** : Triggered by the **Guard Loop** when sensor readings cross defined escalation thresholds.
- **MODEL_RESPONSE_*** : A family of events returning data from user-space daemons (Chat, Vision, etc.) to the logic shell.

3. Deterministic Routing: Interrupt Vectors and the tupleEngine.mjs

To eliminate the "hallucinatory routing" prevalent in cloud-AI products, Project Tricorder implements a deterministic "**tuple-algebra**" **intent engine** within **tupleEngine.mjs**. This engine functions as a software-defined interrupt vector table, resolving input seeds (button presses or wake words) into a definitive, non-probabilistic route: **(Mode, InputChannel, TargetModality, Action)**. The engine utilizes a "first-match wins" logic, ensuring that critical system commands—such as a request for **MEDICAL** search or an emergency **KLAYXON** alert—trigger predictable state transitions. The **config/tricorder.config.json** acts as a **Software-Defined BIOS**, allowing operators to hot-load routing rules and keyword hints for the on-device **SpeechRecognizer** (referenced in **DictationStore.kt**) without recompiling the core kernel.

4. Process Isolation: Termux Loopback Microservices

Project Tricorder isolates high-resource AI models into independent process spaces within Termux. This prevents memory-intensive operations from interfering with real-time sensor polling.

Microservice Port Mapping

Per **BUILD.md** and **MainActivity.kt**, the following ports are reserved:

Port	Service Function
8080	Gemma Multimodal Chat and Reasoning
8081	SmolVLM Vision-Language Processing (Captions)
8188	SD Turbo "Imagine" (Image Generation)
5000	Kokoro Neural Text-to-Speech (TTS)

The **payloadBuilder.mjs** module provides **Protocol Translation**, abstracting the specific requirements of each daemon (e.g., **OpenAI-style chat** vs. **raw vision streams**). If a primary route like **SD Turbo** (Port 8188) is unavailable, the **mcpClient.mjs** logic handles the fallback, rebuilding the payload from scratch for an alternative route (like chat-vision) to ensure the system never transmits malformed data.

5. Multi-Rate Real-Time Scheduling and Clock Synchronization

The system employs a multi-rate scheduling hierarchy to manage disparate telemetry streams, extracted from the logic in **ContextWatch.kt** and **MainActivity.kt**.

GNSS Clock Interpolation

A significant technical challenge is the **GNSS Clock** synchronization. The system utilizes a 1ns resolution GNSS clock, but because the Android UI cannot refresh at that rate, the Kernel

interpolates the time for a consistent **4Hz display update** . This ensures nanosecond-level accuracy for data logging while maintaining UI fluidness.

Polling Rates and Escalation

- **High-Frequency:** IMU updates (120ms limit), GNSS Clock (1ns resolution / 4Hz display).
- **Medium-Frequency:** Flight overhead polling (45s), Citizen alerts (120s).
- **Low-Frequency:** NOAA weather alerts (180s), Earth link probes. In **Sentry Mode** , the **sensorLoopDriver.mjs** runs a low-power "Guard Loop." It only escalates to high-power vision models (**SmolVLM**) when motion thresholds—calculated via a pixel-diff algorithm—cross the **escalation_threshold_pct** defined in the BIOS.

6. The Knowledge Subsystem: Local Caching and Fallback Logic

The knowledge banks (**Memory Alpha** and **MEDICAL**) are implemented as **User-Space Libraries** within **KnowledgeStore.kt** . Unlike the primary AI daemons, these do not require a dedicated loopback port, as they function as direct library services.

Multi-Tier Fallback Strategy

1. **Local JSON Cache:** First-look in **files/knowledge/articles** .
2. **Simple English Wikipedia:** REST summaries for general inquiry.
3. **English Wikipedia (Query Extract):** A critical pivot logic. Since **mdwiki.org** (the MEDICAL catalog) REST summaries are Cloudflare-blocked from direct device access, the system uses mdwiki for titles but pivots to English Wikipedia query extracts for body text and photography.

7. Network and Mesh Topology: Earth Link and Nearby Connections

Project Tricorder operates on a dual-stack networking philosophy, distinguishing between wide-area access and local peer-to-peer survival.

Earth Link

Verified via a successful ping to 8.8.8.8, the **Earth Link** is the gateway for wide-area synchronization, used for fetching Keplerian TLE packs, Space Weather (SOL) data, and model updates for kernel-resident classifiers.

Nearby Link (Camp Link)

Implemented in **NearbyLink.kt** , the **Camp Link** is an opt-in mesh network using Google Play Services Nearby Connections. This link facilitates the exchange of **JSON Log Rows** (from **FieldLog.kt** and **AudioLog.kt**) between peer devices. This decentralized topology allows for real-time alert propagation and log sharing in "blackout" environments without a central router.

8. Conclusion: The Edge-AI OS Paradigm

Project Tricorder Mk I represents a definitive technical synthesis of classic operating system theory and modern machine learning. By treating AI models as isolated user-space daemons,

sensor inputs as hardware interrupt vectors, and the native bridge as a privileged HAL/SCI, the Mk I achieves a level of modularity and reliability impossible in cloud-dependent "AI" products. The architecture's strength lies in its modularity: the ability to swap models, data providers, and routing rules via a single configuration BIOS while maintaining a rock-solid, deterministic core. This makes the Mk I not merely an application, but a true multimodal Edge-AI operating system.